

category: dotnet-general

tags: [enum, refactor, circular-reference, project-reference, LogEventVerify, iaccess-v2]

severity: medium

created: 2026-07-29

updated: 2026-07-29

project-origin: App-Access-V2 (KZTEK iAccess v2)

Đổi field int → enum trong entity dùng chung nhiều project: kiểm tra circular ProjectReference TRƯỚC khi chọn namespace enum

Tình huống gặp phải

User yêu cầu đổi `LogEventVerify.EventAddr/Event/InOutStatus/VerifyType` (đều đang là `int/string`) sang enum cho rõ nghĩa. `LogEventVerify` nằm trong project `Kztek.Object.Entity` (project lỗi, được nhiều project khác reference: `IAccessv2.Objects`, `iAccessDesktopv2.Controller`, `iAccess.Devices.*...`).

Vấn đề / Phát hiện

1. Field `VerifyType` đã có sẵn enum `EmVerifyType` ngay trong `Kztek.Object.Entity.Enums` — đổi type an toàn, không vướng gì.
2. Field `Event`: enum tương ứng `EmTypeEvent` lại nằm ở project **khác** — `IAccessv2.Objects.Objects.Event.EventModel.cs` (project `IAccessv2.Objects`). Muốn `LogEventVerify.Event` có type `EmTypeEvent` thì `Kztek.Object.Entity` phải reference `IAccessv2.Objects` — nhưng `IAccessv2.Objects.csproj` đã có `<ProjectReference Include="..\Kztek.Object.Entity\... → circular reference`, .NET không build được.
3. **Giải pháp**: di chuyển (cắt/dán) định nghĩa enum `EmTypeEvent` từ `IAccessv2.Objects` SANG `Kztek.Object.Entity.Enums` (cùng file với `EmVerifyType`) — đúng hướng phụ thuộc sẵn có (project lỗi không nên phụ thuộc ngược lên project tiêu thụ nó).
4. Sau khi move, kiểm tra lại **toàn bộ** file consumer đang dùng `EmTypeEvent` qua `using IAccessv2.Objects.Objects.Event;` — may mắn là tất cả các file đó (grep xác nhận ~15 file) đã có sẵn `using Kztek.Object.Entity.Enums;` (vì đều cũng dùng `EmVerifyType`), nên không cần thêm using nào, chỉ cần xóa enum khỏi chỗ cũ.
5. **Cạm bẫy tinh vi khác về semantics**: field `Event` không thuần là "Grant/Deny" — nhiều driver (E02/E05/ELV485) gán RAW CODE của hãng (1=thẻ lạ, 27=hộp lệ) trực tiếp vào field này (ngược nghĩa với `EmTypeEvent.Grant=1`), trong khi nơi đọc (`DataServicev8.cs`) lại giả định `Event==1` luôn là "hộp lệ". Đây là 2 domain giá trị xung đột tồn tại sẵn trong code — KHÔNG được tự ý "sửa cho đúng" khi chỉ được yêu cầu đổi kiểu dữ liệu; phải hỏi user trước (ở đây user chọn "chỉ đổi type, giữ nguyên giá trị" — dùng cast `(EmTypeEvent) (rawInt)` để giữ nguyên hành vi).

Giải pháp

```
// TRƯỚC (2 project khác nhau, không thể reference ngược):  
// Kztek.Object.Entity/ZktecoV5L/LogEventVerify.cs  
public int Event { get; set; } // enum EmTypeEvent định nghĩa ở IAccessv2.Objects –  
KHÔNG dùng được ở đây  
  
// SAU: move enum về đúng project lỗi  
// Kztek.Object.Entity/Enums/ZktecoV5L.cs  
public enum EmTypeEvent { Deny, Grant }  
  
// LogEventVerify.cs  
public EmTypeEvent Event { get; set; }  
  
// Consumer có raw code khác domain – giữ nguyên giá trị bằng cast, KHÔNG tự sửa logic:  
log.Event = (EmTypeEvent)(isAccessGrantEvent ? 1 : 27); // parity nguồn E05Net:122,  
KHÔNG phải Grant/Deny thật
```

Áp dụng lại (How to reuse)

- Trước khi đổi 1 field từ `int/string` sang 1 enum có sẵn, luôn kiểm tra **enum đó định nghĩa ở project nào** so với project chứa field — nếu project chứa field ở "tầng thấp hơn" (được nhiều project khác reference) mà enum lại ở "tầng cao hơn", phải move enum xuống tầng thấp trước, không thể `using` xuyên ngược.
- Sau khi move enum, grep toàn repo `using <namespace cũ của enum>` + tên enum để xác nhận mọi consumer vẫn resolve được — ưu tiên tái dùng `using` đã có sẵn (thường consumer đã có `using` tới namespace mới nếu cùng dùng enum khác trong namespace đó).
- Khi 1 field lưu giá trị từ NHIỀU driver/nguồn khác nhau, phải grep hết TẤT CẢ điểm gán/so sánh field đó trước khi đổi kiểu — rất dễ có driver dùng raw code khác domain với enum "tương chừng" khớp (xem thêm `[[camera-type-int-decoded-with-wrong-enum]]` — cùng pattern "2 bảng mã hoá khác nhau dùng chung 1 field int").
- Nếu phát hiện xung đột domain giá trị đang tồn tại sẵn (không phải do refactor gây ra) → dừng lại hỏi user có muốn fix logic hay chỉ đổi kiểu giữ nguyên hành vi, đừng tự quyết.
- Build LẠI CẢ 2 solution liên quan (ở đây: WinForms gốc + Avalonia port song song) sau khi đổi — 2 codebase port song song rất dễ sót 1 file (ở đây sót `ZktecoPullController.cs` chỉ phát hiện được khi build Avalonia solution, không phải khi build WinForms).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Lỗi circular ProjectReference KHÔNG hiện ra cho tới khi build — trước đó chỉ là "chưa move enum" thì code vẫn hợp lệ; chỉ khi thử `using` enum ở project thấp hơn mới lộ ra lỗi thiết kế.
- ⚠ 2 codebase song song (WinForms `iAccessDesktopv2` + Avalonia port `iAccessDesktopv2.Avalonia`) có nhiều file gần như bản sao của nhau (`E02Net_V3.cs` ↔ `E02NetController.cs...`) — sửa 1 bên dễ quên bên kia; luôn build CẢ HAI sau khi đổi type field dùng chung.
- ⚠ Method nhận tham số kiểu `int` sẵn có (vd `AccessMemoryCollection.GetByUserIdAndType(int userId, int accessKeyType, ...)`) sẽ cần thêm `(int)` cast tại MỌI call site sau khi field đổi sang enum — dễ sót nếu chỉ sửa chỗ gán mà quên chỗ đọc/truyền tham số.

Tham chiếu

- File chính: `iAccessDesktopv2/Kztek.Object.Entity/ZktecoV5L/LogEventVerify.cs`,
`iAccessDesktopv2/Kztek.Object.Entity/Enums/ZktecoV5L.cs`
- Enum di chuyển từ:
`iAccessDesktopv2/IAccessv2.Objects/Objects/Event/EventModel.cs`
- ~18 file consumer cần sửa (grep `log\.Event\s*=\|log\.VerifyType\s*=` để liệt kê đầy đủ)
- Lesson liên quan: `[[camera-type-int-decoded-with-wrong-enum]]`